TP3: Les arbres de décision

Exercice 1: On souhaite créer un arbre de décision en utilisant la bibliothèque scikit-learn pour classifier si une personne jouera au football le dimanche ou non.

1.) Importer les bibliothèques nécessaires : **pandas, numpy, matplotlib, et sklearn.**

La bibliothèque **sklearn** comprend plusieurs fonctions :

- from sklearn.model_selection import **train_test_split**
- from sklearn import **tree**
- from sklearn.tree import **DecisionTreeClassifier**
- from sklearn.tree import **export_text**
- from sklearn.tree import **plot_tree**
- from sklearn.metrics import **confusion_matrix**
- from sklearn.metrics import **ConfusionMatrixDisplay**
- from sklearn.metrics import **accuracy_score**
- from sklearn.metrics import **classification_report**

2.) Importer le dataset ”playFootball.csv”et visualiser son contenu.

3.) Afficher les types de données du dataset avec la variable **dtypes.**.

4.) Afficher les informations du dataset avec la fonction **info()**.

5.) Convertir les données catégoriques (textuelles) en données (0/1, true/false,..). Uti- liser la fonction **get_dummies()**. Afficher le nouveau dataset indicateur.

6.) Préciser **la sortie y** du modèle d'arbre de décision, correspondant à la colonne 'Play' et les **entrées X**, correspondant à toutes les colonnes du

dataset sauf la colonne 'Play'. Utiliser la fonction **drop()**.

7.) Diviser le dataset en 2 parties: **apprentissage (70%** du nombre total des enregis- trements) et **test (30%)**. Utiliser la fonction **train_test_split()**.

8.) Afficher les longueurs des données de la base d'apprentissage et celles de test.

9.) Définir le modèle de l'arbre de décision. Utiliser la fonction **'DecisionTreeClassi- fier()'**. Préciser **'entropy'** comme critère de classification, et 2 comme profondeur maximale de l'arbre.

10.) Faire l'entrainement de l'arbre sur les données de la base d'apprentissage en utili- sant le modèle de l'arbre de décision et la fonction **'fit()'**.

11.) Calculer les prédictions sur les données d'entrées de la base de test en utilisant le modèle de l'arbre de décision et la fonction **'predict()'**.

12.) Faire l'évaluation du modèle avec **la matrice de confusion**, et **la précision** (accu- racy).

13.) Afficher la matrice de confusion et la précision.

14.) Afficher le rapport de classification avec la fonction **classification_report().** (facul- tatif)

15.) Afficher la représentation textuelle de l'arbre. Utiliser la fonction **export_text()**.

16.) Visualiser le diagramme de l'arbre de décision sur une figure. Utiliser la fonc- tion **plot_tree()** avec les paramètres suivants: dtree, feature_names=X.columns, fontsize=12, filled=True, class_names=['Not_Play', 'Play']

Exercice 2: Cette fois ci, on souhaite créer un arbre de décision pour prédire les prix des jeux vidéo en fonction de leurs coût de production.

1.) Importer les bibliothèques nécessaires: pandas, numpy, matplotlib, et sklearn. La bibliothèque sklearn comprend plusieurs fonctions:

- from sklearn.tree import DecisionTreeRegressor

- from sklearn.tree import export_graphviz

2.) Utiliser le dataset suivant
: [['*AssetFlip*', 100, 1000],
['*TextBased*', 500, 3000],
['*V isualNovel*', 1500, 5000],
['2*DPixelArt*', 3500, 8000],
['2*DV ectorArt*', 5000, 6500],
['*Strategy*', 6000, 7000],
['*FirstPersonShooter*', 8000, 15000],
['*Simulator*', 9500, 20000],
['*Racing*', 12000, 21000],
['*RPG*', 14000, 25000],
['*Sandbox*', 15500, 27000],
['*Open − World*', 16500, 30000],
['*MMOFPS*', 25000, 52000],
['*MMORPG*', 30000, 80000]]

3.) Charger le dataset avec la fonction array de numpy. Chaque enregistrement com- prend le **nom** du jeux vidéo, son **coût** de production et son **prix** de vente. Visualiser le dataset.

4.) Préciser la sortie y du modèle d'arbre de décision, correspondant à la colonne 'prix de vente' et l'entrée X, correspondant à la colonne 'coût de production'.

5.) Définir le modèle de l'arbre de décision de régression. Utiliser la fonction **'Decisi- onTreeRegressor()'**.

6.) Faire l'entrainement de l'arbre sur les données de la base d'apprentissage.

7.) Calculer les prédictions sur les coûts de production suivants : 3750.0, 18000.0, et 20000.0 en utilisant le modèle de l'arbre de décision et la fonction 'predict()'. Afficher les prix de vente prédits.

8.) Tracer une courbe qui montre la relation entre le coût de production du jeux vidéo et son prix de vente. Utiliser les fonctions scatter(), plot(), title(), xlabel(), ylabel(), show() et export_graphviz().